Module 6: Prototyping vs Production

Speed, quality, and operational readiness

Module 6: Prototyping vs Production

Primary sources: Osmani, Chapter 6; Thomas & Hunt, Topics 10-11 and Topic 13: Prototypes and Post-it Notes.

Learning Outcomes

- Distinguish prototype and production systems.
- Analyze tradeoffs between speed and reliability.
- Identify gaps between generated code and production needs.
- Explain strategies for evolving prototypes into production systems.

Prototype Value

AI is strong at rapid prototyping because it can quickly create visible progress.

Prototype goals:

- Learn feasibility.
- Expose requirement gaps.
- Compare alternatives.
- Get feedback early.

Prototype risk: mistaking a learning artifact for a production system.

Prototype Tool Tradeoffs

Osmani frames AI prototyping tools around two tensions.

- Fidelity: how closely the output matches the idea, mockup, or prompt.
- Control: how much you can guide architecture, code structure, and changes.

High-speed tools can validate ideas quickly, but low control can leave code that is hard to inspect, integrate, or harden.

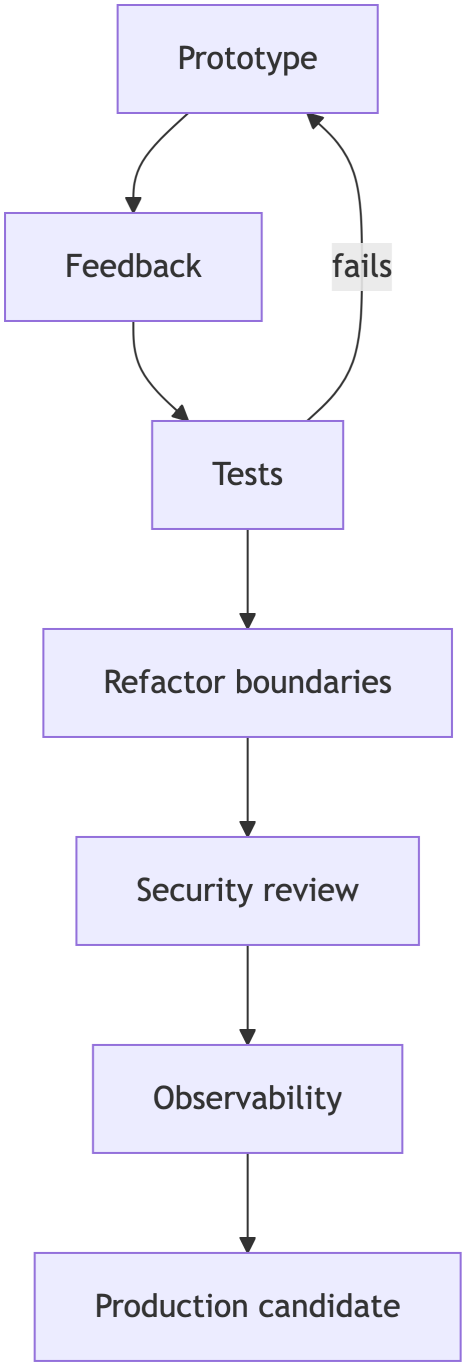
Iterative Refinement

AI prototyping works best as a fast feedback loop.

1. Generate a baseline.
2. Run or inspect it.
3. Refine the prompt with specific changes.
4. Check whether the new version still supports the prototype goal.
5. Stop when you have learned enough.

Iteration is useful only if it answers the prototype question instead of drifting into accidental product scope.

Prototype to Production Path



Production Means Constraints

Production-ready software must address:

- Reliability.
- Security.
- Observability.
- Maintainability.
- Dependency management.
- Error recovery.
- Documentation.
- Operational cost.

Prototype to Production Gap

AI-generated prototypes often omit:

- Input validation.
- Permission boundaries.
- Logging discipline.
- Test coverage.
- Failure modes.
- Deployment assumptions.
- Data migration concerns.

Evolving Toward Production

Osmani's production transition is a mindset shift from speed to discipline.

- Review architecture and split prototype code into maintainable modules.
- Add error handling and edge cases beyond the sunny-day path.
- Check performance, security, and dependency choices.
- Add documentation for future maintainers.
- Build tests before and during refactoring.
- Replace temporary shortcuts with real integrations.

Prototype Traps

Two common traps:

- Scope creep: AI makes adding "one more feature" too cheap.
- Fake integration: mock data, local files, or console logs hide real production work.

Keep a visible list of temporary choices and production TODOs so the prototype does not accidentally become the architecture.

Orthogonality

Orthogonal components can change independently. Thomas and Hunt's test: if a requirement changes, how many modules are affected?

For the Java agent project:

- Prompt construction should not depend on CLI parsing.
- Tool dispatch should not depend on UI output.
- Evaluation should run without live model calls.
- Secret handling should be isolated.

Reversibility

Decisions should be easy to change when uncertainty is high.

Thomas and Hunt's warning: critical decisions shrink the target over time. Treat uncertain choices as written in sand, not carved in stone.

Use AI to explore reversible options:

- Interfaces before implementations.
- Adapter boundaries around model providers.
- Configurable paths and tools.
- Small commits that can be discarded.

Production Readiness Questions

- What can fail?
- How will we know?
- What is the safe default?
- What should be logged?
- What must never be logged?
- What requires human approval?

Lab 3 Final Connection

Lab 3 implementation should prove that your system is controlled, not just functional.

Evidence should include:

- Required Java tools.
- Sandbox enforcement.
- Inspectable output.
- Architecture notes.
- Evaluation results.

Practice Activity

Take a working prototype feature and label each part:

- Keep for production.
- Rewrite before production.
- Add tests.
- Add monitoring.
- Add security review.

Takeaways

- AI speed is useful for learning, not a substitute for readiness.
- Orthogonal design makes generated code safer to evolve.
- Reversibility reduces the cost of AI-assisted experiments.